

Module 3

This is a single, concatenated file, suitable for printing or saving as a PDF for offline viewing. Please note that some animations or images may not work.

Module 3 Study Guide and Deliverables

January 30 – February 5

Theme:	Introduction to Crypto
Topics:	• The role and property of crypto • Terminology: Alice, Bob, Eve, encrypt, decrypt, cryptography, cryptanalysis • Classical encryption: Caesar Cipher, ROTx, substitution cipher • Symmetric encryption: DES, AES • Key negotiation: DH • Asymmetric encryption: RSA • Hash: MD, MAC, HMAC • Data authenticity and confidentiality
Readings:	• Chapter 2.3
Assignments:	• Lab 3 due Tuesday, February 6 at 6:00 AM ET (Access at "Assignments" on the left-hand course menu.)
Assessments:	• Quiz 3 due Tuesday, February 6 at 6:00 AM ET (Access at "Assessments" on the left-hand course menu.)
Discussions:	• None
Live Classroom:	Join the live classroom and the facilitator's live office hour session at "Live Classroom/Offices" > "Live Classroom" > Launch Meeting. • Lecture: Tuesday, January 30, 7:30-9:00 PM ET • Facilitator office hour: TBD

Topic 1 Encryption

Learning Outcomes

By the end of this module, you will be able to:

- Describe basic concepts related to cryptography such as symmetric key encryption, asymmetric key encryption, stream cipher, block cipher, cryptanalysis, one-time pad, message digest, MAC, HMAC, DES, AES, MD4/5, SHA1/2/3, digital signature, etc.
- Understand the simple substitution cipher and the cryptanalysis against it.
- Understand how Diffie-Hellman can be used to generate a shared secret.
- Understand how RSA can be used to generate a pair of public key and private keys.
- Know how to use symmetric key encryption and asymmetric key encryption to ensure data authenticity and confidentiality.

Problems Addressed by Cryptography

Cryptography means “secret writing” in the context of transmitting messages. Consider two parties, Alice and Bob, a sender and a receiver, who would like to transmit messages through a public communication channel, T. If a third-party, Eve, wants to access the messages transmitted via T (to read, change, or even destroy them), we call Eve an adversary or intruder. (Cryptography literature often uses Alice for the sender, Bob for the intended recipient, and Eve for the adversary.) Whenever Alice sends messages to Bob via T, they are vulnerable to exploitation, and Eve may try to access them in any of the following ways

- drop them, by preventing the messages from reaching Bob, thereby compromising the availability of the messages
- intercept them, by reading the messages, thereby compromising the confidentiality of the messages
- modify them, by revising the messages in some way, thereby compromising the integrity of the messages
- fabricate them, by producing a legitimate-looking message and sending it to Bob, thereby also compromising the integrity of the messages

As we can see, Eve may compromise the CIA of the messages between Alice and Bob. Fortunately, cryptography can help to address all of the above problems. Below, we will introduce the fundamentals of cryptography and learn how it is used to securely transmit messages via insecure communication channels.

Terminology

Fundamentals

Plaintext is the original message in intelligible form, which is interpretable to humans. **Encryption** is the process of converting plaintext into unintelligible form, called **ciphertext**, which looks like a random data stream, uninterpretable to humans. **Decryption** is the reverse of encryption, that is, converting the ciphertext back to plaintext. A **cipher** consists of encryption and decryption algorithms, which are the rules used to encrypt the plaintext and decrypt the ciphertext. The encryption and decryption algorithms may involve a **key**, usually a string of characters only known to the sender and receiver. Hence, the resulting ciphertext depends on the original plaintext, the encryption algorithm, and the key. This dependency can be formally written as $C = E(K, P)$, where E is the encryption algorithm, C denotes the ciphertext, K is the key, and P denotes the plaintext. Correspondingly, decrypting from ciphertext to plaintext is written as $P = D(K, C)$, where D is the decryption algorithm.



Encryption and decryption using a key

Cryptography and Cryptanalysis

Cryptography is the art (and sometimes the science) of secret writing by creating ciphers to communicate. It is probably the strongest defense in the arsenal of computer security protection. It can also be used to guarantee other properties, e.g., authenticity of data, non-repudiation, etc., as we will learn later in this module. In contrast, **cryptanalysis** is used to break the ciphers, i.e., by inferring the plaintext from the ciphertext without knowing the key. The history of cryptography is an arms race between cryptographers and cryptanalysts, with the former trying to design a strong cipher and the latter trying to break it. Any cipher is theoretically breakable, and if given enough time and data (pairs of plaintexts and ciphertexts), cryptanalysts can learn the plaintext of unseen ciphertext. But, many are impractical to break.

Consider breaking a ciphertext with 25 characters, encrypted by substituting one character in plaintext with another. In total, there exist 26^{25} (approximately 10^{35}) possible decipherments, since there are 26 letters in the English alphabet and each character in plaintext could be any of the 26. A **brute force** cryptanalysis would try all possibilities to find the right one out of 26^{25} . If you have a quite powerful computer capable of performing 10^{20} operations per second (faster than the most powerful supercomputer in the world currently), breaking such a cipher would take 10^{15} seconds, or roughly 10^{10} years. In this case, although we know such a cipher is theoretically breakable, we say this is **computationally hard** to break based on existing technology. Note that the security of the cipher should depend on the key, rather than the algorithms. The encryption/decryption algorithms are public, while the keys are only known to the communicants to ensure others, without knowing the key, cannot (because it is computationally hard) decrypt the ciphertext.

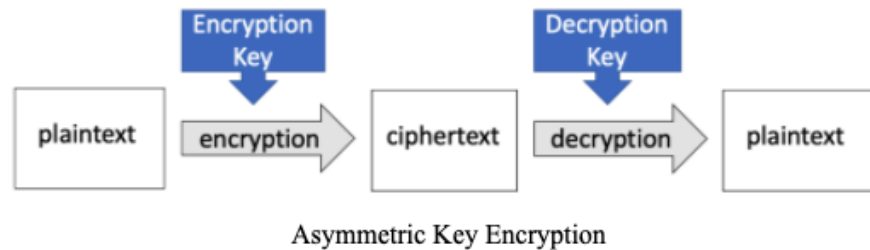
Symmetric and Asymmetric Encryption

Generally, there are two kinds of encryption, symmetric key encryption and asymmetric key encryption. For the former, the sender, Alice, and receiver, Bob, share a secret, which is the key. Alice uses the key to encrypt the plaintext message into ciphertext $C=E(K, P)$. After receiving the ciphertext, Bob uses the key to decrypt the ciphertext, thus deriving the original plaintext $P=D(K, C)$, i.e., $P=D(K, E(K, P))$.



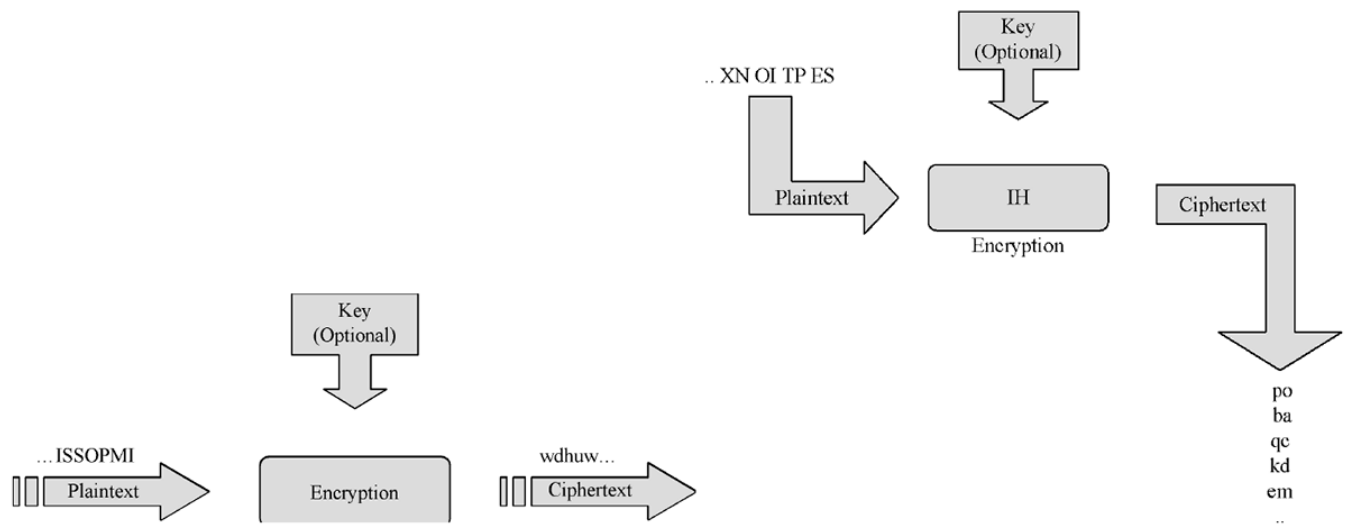
Symmetric Key Encryption

In asymmetric key encryption, the encryption key and decryption key come in pairs, but differ, denoted as KE and KD respectively. Alice encrypts the plaintext message using KE to obtain the ciphertext $C=E(KE, P)$. After receiving the ciphertext, Bob uses the decryption key to decrypt the ciphertext, thus deriving the original plaintext $P=D(KD, C)$, i.e., $P=D(KD, E(KE, P))$.



Stream and Block Ciphers

Depending on the nature of the data to be encrypted, there may exist a stream cipher and a block cipher. A stream cipher encrypts one bit or one byte at a time, while a block cipher encrypts a fixed number of bits as a single chunk.



Source: (Pfleeger, et al., 2015)

Classic Cryptography

Cryptography is not new. In fact, it has been used for centuries. Below, we will introduce several examples of classic encryption algorithms, including the Caesar cipher and the simple substitution cipher, as well as the one-time pad, and we will perform cryptanalysis on them; that is, we will try to break them.

Caesar Cipher

The Caesar cipher, invented by Julius Caesar, is a rotational cipher, where each letter in plaintext is rotated left or right by some number of positions to get the ciphertext. For example, simply take each letter in the plaintext message and shift it to the third letter after it in the alphabet. Then, S in the plaintext becomes V in the ciphertext. (Here we assume the alphabet is cyclic, i.e., A is after Z.) Applying this procedure to the entire plaintext yields the ciphertext.

Caesar Cipher Table

Plaintext	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	X	Y	Z
Ciphertext	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	X	Y	Z	A	B	C

Test Yourself

Based on the Caesar cipher table above, what is the ciphertext of the message below, and how do you decrypt the ciphertext?

SECURITYANDPRIVACY

VHFXULWBDQGSULYDFB

Decrypt by shifting each letter in the ciphertext 3three positions left.

Another example of a Caesar cipher is **ROT13**. ("rotate by 13 places"), which replaces a letter with the 13th letter after it in the alphabet. ROT13 is used in online forums as a means of hiding spoilers, punchlines, puzzle solutions, and offensive materials from the casual glance.

Suppose we replace each letter with a number as follows:

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12
n	o	p	q	r	s	t	u	v	w	x	y	z
13	14	15	16	17	18	19	20	21	22	23	24	25

The encryption algorithm of the Caesar cipher above can be expressed as below to obtain a letter, c, in ciphertext from a letter, p, in plaintext:

$$c = (p+3) \bmod 26$$

where mod is the modulo operation (obtaining the remainder of a division, e.g., $11 \bmod 7 = 4$). To be generic for rotating K letters, the encryption algorithm of the Caesar cipher can be written as:

$$C = E(K, P) = (P+K) \bmod 26$$

where K is an integer ranging from 1 to 25. The corresponding decryption algorithm is:

$$P = D(K, C) = (C-K) \bmod 26$$

If we know the given ciphertext is obtained using a Caesar cipher, cryptanalysis is quite easy, i.e., use brute force for all the possible keys. This is feasible because: (1) the key space is limited, with only 25 possibilities in total, and (2) the plaintext message should be interpretable.

Cryptanalysis of Ceasar Using Brute Force

KEY	PHHW PH DIWHU WKH WRJD SDUWB
1	oggv og chvgt vjg vqic rctva

```

2   nffu nf bgufs uic uphb qbsuz
3   meet me after the toga party
4   ldds ld zesdq sgd snfz ozqsx
5   kccr kc ydrpc rfc rmey nyprw
6   jbbq jb xcqbo qeb qldx mxoqv
7   iaap ia wbpap pda pkcw lwnpu
8   hzzo hz vaozm ocz ojbv kvmot
9   gyyn gy uznyl nby niau julns
10  fxxm fx tymxk max mhzt itkmr
11  ewwl ew sxlwj lzw lgys hsjlq
12  dvvk dv rkwvi kyv kfxr grikp
13  cuuj cu qvjuh jxu jewq fqhjo
14  btti bt puitg iwt idvp epgin
15  assa as othsf hvs hcuo dofhm
16  zrrg zr nsgrg gur gbtn cnegi
17  yqqf yg mrfqd ftq fasm bmdfk
18  xppe xp lqepc esp ezrl alcej
19  wood wo kpdob dro dyqk zkbdi
20  vnnc vn jocna cqn cxpj yjach
21  ummb um inbmz bpm bwoi xizbg
22  tlla tl hmaly aol avnh whyaf
23  skkz sk glzxx znk zumg vgxze
24  rjyy rj fkyjw ymj ytlf ufwyd
25  qiix qi ejxiv xli xske tevxc

```

Source: (from Stallings, 2011)

Simple Substitution Cipher

A simple substitution cipher replaces each letter in the plaintext for a different letter in the alphabet. In order for the decryption to work well, there should be no two plaintext letters replaced by the same ciphertext letter. Otherwise, the decryption will be confused due to multiple possible plaintexts. Creating a table with the plaintext letters in the top row and the corresponding ciphertext letters below is very convenient when performing encryption.

Simple Substitution Encryption Table

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
C	I	S	Q	V	N	F	O	Q	A	X	M	T	G	U	H	P	B	K	L	R	E	Y	D	Z	J

Source: (Hoffstein, et al., 2014)

Switching the two rows of the above table, we will get the corresponding decryption table. However, decryption based on such a decryption table is a little inefficient, since the ciphertext letters are not in alphabetical order. Therefore, it is better to reorganize the decryption table based on the alphabetical order of the ciphertext letters as below.

Simple Substitution Decryption Table

j	r	a	x	v	g	n	p	b	z	s	t	l	f	h	q	d	u	c	m	o	E	i	k	w	y
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

Source: (Hoffstein, et al., 2014)

Test Yourself

Based on the simple substitution encryption and decryption tables above, what is the ciphertext of the below message, and how do you decrypt the ciphertext? S E C U R I T Y A N D P R I V A C Y

K V S R B W L Z C G Q H B W E C S Z

By referring to the decryption table, the ciphertext can be decrypted back to
S E C U R I T Y A N D P R I V A C Y

For a simple substitution cipher, each letter is replaced by another and no two letters are replaced by the same letter. Hence, there will be a total of 25! (factorial operation, e.g., $5! = 5 * 4 * 3 * 2 * 1$) possibilities, or roughly $1.6 * 10^{25}$, which is also computationally hard based on what we illustrated before. However, this does not mean that the simple substitution cipher is immune from attacks. If we know some characteristics of the underlying plaintext, e.g., that it is written in English without compression, we are still able to break it. Consider the ciphertext below:

A Segment of Ciphertext

UZQSOVUOHXMOPVGPOZPEVSGZWSZOPRPESXUDBMET SXAIZ

VUEPHZHMDZSHZOWSFPAPDTSVPQUZWYMXUZHUSX

EPYEPDPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ

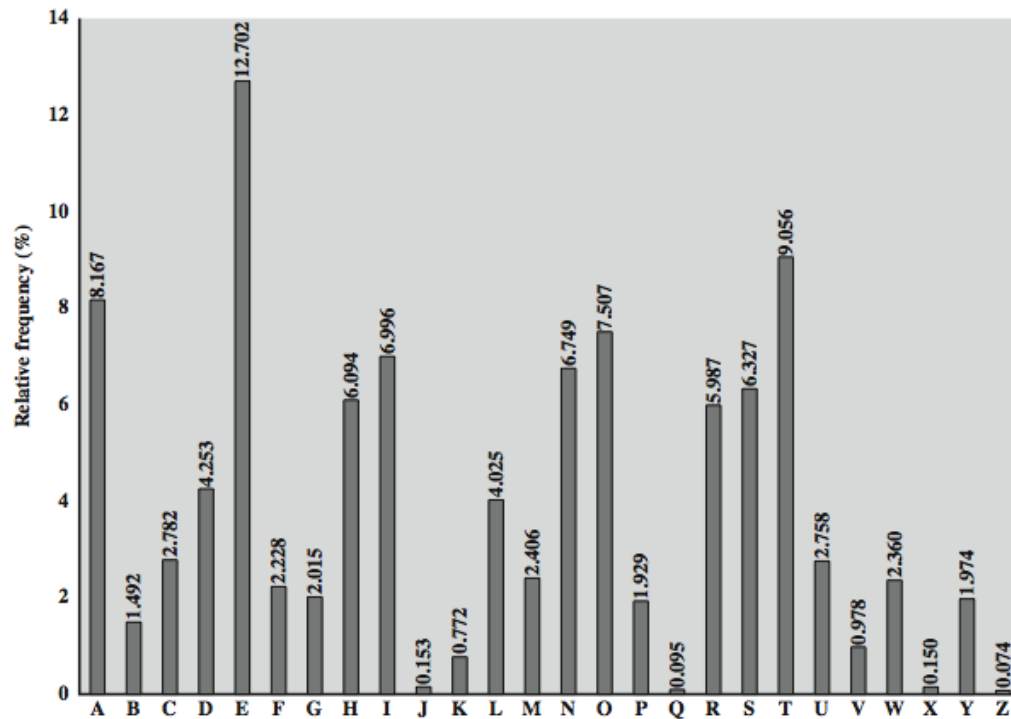
Source: (Stallings, 2011)

First, we summarize the relative frequency of each letter that appears in the above ciphertext, and compare it with the relative frequency of English letters based on statistics. If the ciphertext is long enough, such comparison results in a relatively accurate match between letters in ciphertext and those in plaintext. However, if the ciphertext is not long enough, we cannot obtain an accurate match. Through comparison, we learn that the letters P and Z in the ciphertext might be E and T in plaintext, but we are not sure if it is P to E and Z to T or P to T and Z to E. Furthermore, the relative frequency of letters S, U, O, M, and H is also high, and those letters might each correspond to one of the letters A, H, I, N, O, R and S in plaintext. Meanwhile, the letters in the ciphertext with low relative frequency might each correspond to one of the letters B, J, K, Q, V, X, and Z in plaintext.

Relative Frequency of Each Letter in the Ciphertext

P	13.33	H	5.00	F	3.33	B	1.67	C	0.00
Z	11.67	D	5.83	W	3.33	G	1.67	K	0.00
S	8.33	E	5.00	Q	2.50	Y	1.67	L	0.00
U	8.33	V	4.17	T	2.50	I	0.83	N	0.00
O	7.55	X	4.17	A	1.67	J	0.83	R	0.00
M	6.67								
Source: (Stallings, 2011)									

Relative Frequency of English Letters based on Statistics



Source: (Stallings, 2011)

What we can do now is try some possible replacements to see if we can get an outline of the plaintext message. It is quite useful to consider the combination of two letters, e.g., one of the most frequent combinations of two letters is "th." In the ciphertext, the most frequent combination of two letters is ZW, which appears three times. Hence, it is highly possible that Z corresponds to plaintext T and W to plaintext H. Together with our previous analysis, we can determine that P likely corresponds to plaintext E. Then, we notice ZWSZ in the first line of ciphertext should correspond to TH_T, so we guess S might correspond to plaintext A.

Partial Decryption of the Ciphertext

```
UZQSOVUOHXMOPVGPOZPEVSGZWSZOPRPEXUDBMET SXAIZ
t a e e t e a t h a t e e a a t
```

```
VUEPHZHMDZSHZOWSFPAPDTSVPQUZWYMXUZHUSX
e t t a t h a e e e a e t h t a
```

```
EPYEPDPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ
e e e t a t e t h e t
```

Source: (Stallings, 2011)

We can continue such analysis, and finally, it is not difficult to obtain the plaintext message, as below:

Final Plaintext Message

```
it was disclosed yesterday that several informal but
direct contacts have been made with political
representatives of the viet cong in moscow
```


Overall, the simple substitution cipher does not hide the underlying frequency of characters, thus it is still breakable.

One-time pad (OTP)

Before we introduce the one-time pad, it is useful to review the operation of exclusive or, i.e., XOR. It is a Boolean (values are either true or false, i.e., 1 or 0) binary (only two inputs) operation that is true when only one of two inputs is true. The operator is written as $\oplus$. The rules of the XOR operation are as below:

$$0 \oplus 0 = 0, 1 \oplus 0 = 1, 0 \oplus 1 = 1, 1 \oplus 1 = 0$$

There are several properties of XOR operation:

- Commutative. Operands can be flipped around: $a \oplus b = b \oplus a$
- Associative. XOR can be applied in any order: $a \oplus (b \oplus c) = (a \oplus b) \oplus c$
- Any bit XOR itself is 0: $a \oplus a = 0$
- Any bit XOR 0 is itself: $a \oplus 0 = a$

Test Yourself

Given the above rules, how can we derive: $a \oplus b \oplus a = b$?

$$a \oplus b \oplus a = a \oplus a \oplus b \text{ (associative)}$$

A one-time pad encrypts the plaintext using a random secret key (referred to as a pad because it was originally recorded on a physical, paper pad) with the same length as the plaintext. Each bit in the plaintext is XORed with the corresponding bit in the key to produce the bit in the ciphertext. A one-time pad is unbreakable if the key is truly random (unpredictable by attackers) and only used once. Theoretically, the one-time pad has the strongest security guarantee, i.e., attackers cannot derive the plaintext from the ciphertext without knowing the key. However, in reality, it is hard to use widely for two reasons. First, it is nontrivial to produce a large amount of truly random keys, each of which is used only once and then discarded. Second, key distribution and protection is challenging for Alice and Bob.

Think about what happens if the key is used twice in a one-time pad.

We have plaintext p_1 and its ciphertext c_1 , plaintext p_2 and ciphertext c_2 . Both of them are using the same key, k . The attacker knows c_1 and c_2 , so she performs $c_1 \oplus c_2$:

$$c_1 = (p_1 \oplus k), c_2 = (p_2 \oplus k)$$

$$c_1 \oplus c_2 = (p_1 \oplus k) \oplus (p_2 \oplus k)$$

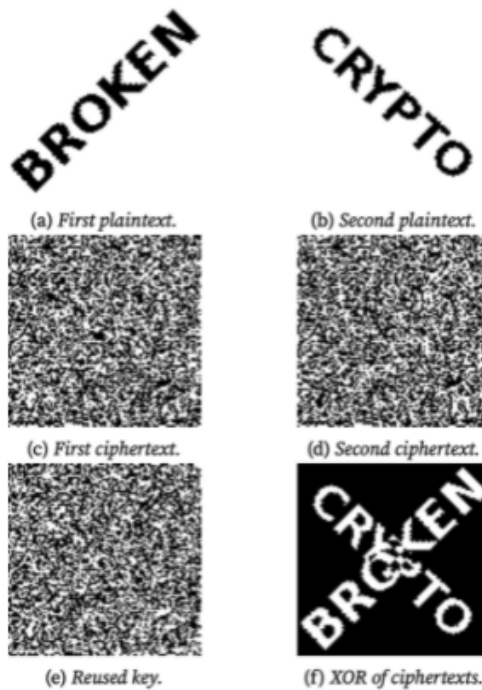
$$= p_1 \oplus k \oplus p_2 \oplus k$$

$$= p_1 \oplus p_2 \oplus (k \oplus k)$$

$$= (p_1 \oplus p_2) \oplus 0$$

$$= p_1 \oplus p_2$$

So, it still looks like the attacker cannot learn either p_1 or p_2 from $p_1 \oplus p_2$. Below is a diagram illustrating what the attacker can learn from $p_1 \oplus p_2$.



Two plaintexts, the reused key, their respective ciphertexts, and the XOR of the ciphertexts. Information about the plaintexts clearly leaks through when we XOR the ciphertexts (from Van Houtven, 2017).

Note, the Caesar cipher, simple substitution cipher, and one-time pad are all symmetric key encryptions, since they all use the same key to encrypt and decrypt.

Modern Cryptography

Modern cryptography relies on deep mathematics. We will introduce the basics of them and how they can be used (applied cryptography), rather than mathematical proof of them.

DES and AES

The Data Encryption Standard (**DES**) was developed by IBM in 1972 for the U.S. National Institute of Standards and Technology (NIST), later called the National Bureau of Standards (NBS). It signaled the beginning of the modern era of cryptography. DES is a block cipher, encrypting the plaintext input as blocks of 64 bits, i.e., 8 bytes. For each block, 16 iterations of substitution and permutation are performed, with each iteration replacing groups of bits, shuffling the bits, and merging the bits with the key. The key is 64 bits long as well, with 56 bits used in the encryption and the other 8 bits as parity bits to check errors of the key during transmission. Hence, the strength of DES is 56 bits (the length of the key). Refer to (Pfleeger, et. al. 2015) page 779 for a detailed description of the DES algorithm.

As computing power has significantly improved during the last few decades, **DES** has been shown to be breakable, mainly due to the use of the short key. To address this limitation, researchers proposed **Double DES** and **Triple DES** to improve the key strength. Double DES uses two keys, k_1 and k_2 , and performs two rounds of DES respectively, i.e., $E(k_2, E(k_1, d))$, where d is the plaintext data and E indicates the standard DES encryption algorithm. The intent of Double DES was to increase the difficulty of breaking the encryption by making the key strength $56 * 2 = 112$ bits. However, researchers later found that using two encryptions with different 56-bit keys is almost equivalent to using one encryption with one 57-bit key (double the strength of standard DES). Therefore, Double DES only adds a small burden for the attackers attempting to break it. But follow-up research found that three rounds of DES with three different keys can significantly improve the strength. The Triple DES works as $E(k_3, E(k_2, E(k_1, d)))$, giving a strength roughly equivalent to a 112-bit key. A variant of Triple DES uses two keys only, i.e., $E(k_1, D(k_2, E(k_1, d)))$, with a strength roughly equivalent to an 80-bit key. Note that to be compatible with a system that only supports DES, Triple DES with two keys should be used.

Forms DES

Form	Operation	Properties	Strength
DES	Encrypt with one key	56-bit key	Inadequate for high-security applications by today's computing capabilities
Double DES	Encrypt with first key; then encrypt result with second key	Two 56-bit keys	Only doubles strength of 56-bit key version
Two-key Triple DES	Encrypt with first key, then encrypt (or decrypt) result with second key, then encrypt result with first key (E-D-E)	Two 56-bit keys	Gives strength equivalent to about 80-bit key (about 16 million times as strong as 56-bit version)
Triple-key Triple DES	Encrypt with first key, then encrypt or decrypt result with second key, then encrypt result with third key (E-E-E)	Three 56-bit keys	Gives strength equivalent to about 112-bit key about 72 quintillion ($75 * 10^{15}$) times as strong as 56-bit version
Source: (from Pfleeger, et al., 2015)			

The Advanced Encryption System (**AES**) was adopted by NIST in 2001 as the successor of DES. AES is also a block cipher, with each block comprised of 128 bits. The key sizes can be 128, 192, or 256 bits. There are no practical attacks known against AES currently. It has been the commercial-grade encryption algorithm for years. Note that DES, Double DES, Triple DES, and AES are all symmetric key encryption algorithms, since they all use the same key to encrypt and decrypt. Refer to the course textbook (Pfleeger, et al., 2015) page 789, for a detailed description of the AES algorithm.

Diffie-Hellman Key Exchange (DHKE)

The Diffie-Hellman key exchange is a protocol designed by Whitfield Diffie and Martin Hellman in 1976, which allows two communicants to negotiate a secret (i.e., key) over insecure channels. This is the basis for symmetric key encryption to have the initial key setup for the communicants Alice and Bob. Consider that at the very beginning, Alice and Bob do not have any key used to encrypt/decrypt, so they can only send plaintext over the transmission channel, which is vulnerable to Eve's interception. DHKE addresses this problem based on very deep mathematics. We will focus on how Alice and Bob can utilize DHKE to negotiate a key, rather than the underlying math.



At the very beginning, Alice and Bob can negotiate to pick up a large prime number, p , and a base, g , (with $g < p$). Note that both p and g are public, which means Eve also knows them by listening to the communication channel. Then, Alice picks up a private value, x_1 , and Bob picks up a private value, x_2 , and each respectively performs the computation in Step 2 individually to get y_1 and y_2 . Then, they exchange y_1 and y_2 through the channel. Note that here Eve can also intercept y_1 and y_2 . Finally, Alice and Bob can compute the secret, z , in Step 4, which can be used as the key for encryption/decryption.

	Alice	Bob
Step 0	negotiate p and g	negotiate p and g
Step 1	pick a private value $x_1 (< p-1)$	pick a private value $x_2 (< p-1)$
Step 2	generate $y_2 = g^{x_2} \bmod p$	generate $y_2 = g^{x_2} \bmod p$
Step 3	exchange y_1 for y_2	exchange y_2 for y_1
Step 4	generate the secret shared key z $z = y_2^{x_1} \bmod p$	generate the secret shared key z $z = y_1^{x_2} \bmod p$

Recall that when Alice and Bob negotiate the key using DHKE, Even can intercept p , g , y_1 (for Alice), and y_2 (for Bob), but she doesn't know x_1 or x_2 . Looking at Step 2, solving x from $y = g^x \bmod p$, given you know y , g , and p , is computationally hard. For example, if $p = 170141183460469231731687303715884105727$ (i.e., $2^{127} - 1$), it would take roughly $1.14824 \cdot 10^{21}$ steps to solve the problem (with each step requiring many calculations) based on state-of-the-art cryptanalysis. Even using Google's computers, which are estimated to perform 300 trillion calculations per second, it would take roughly five years to solve. Note the above prime p was found in just 1800 seconds.

Test Yourself

Alice and Bob publicly agree to use $p = 23$ and $g = 5$. Alice chooses $x_1 = 4$ and Bob chooses $x_2 = 3$.

What is the key negotiated between them using DHKE?

Step 2:

Alice: $y_1 = g^{x_1} \bmod p = 5^4 \bmod 23 = 4$

Bob: $y_2 = g^{x_2} \bmod p = 5^3 \bmod 23 = 10$

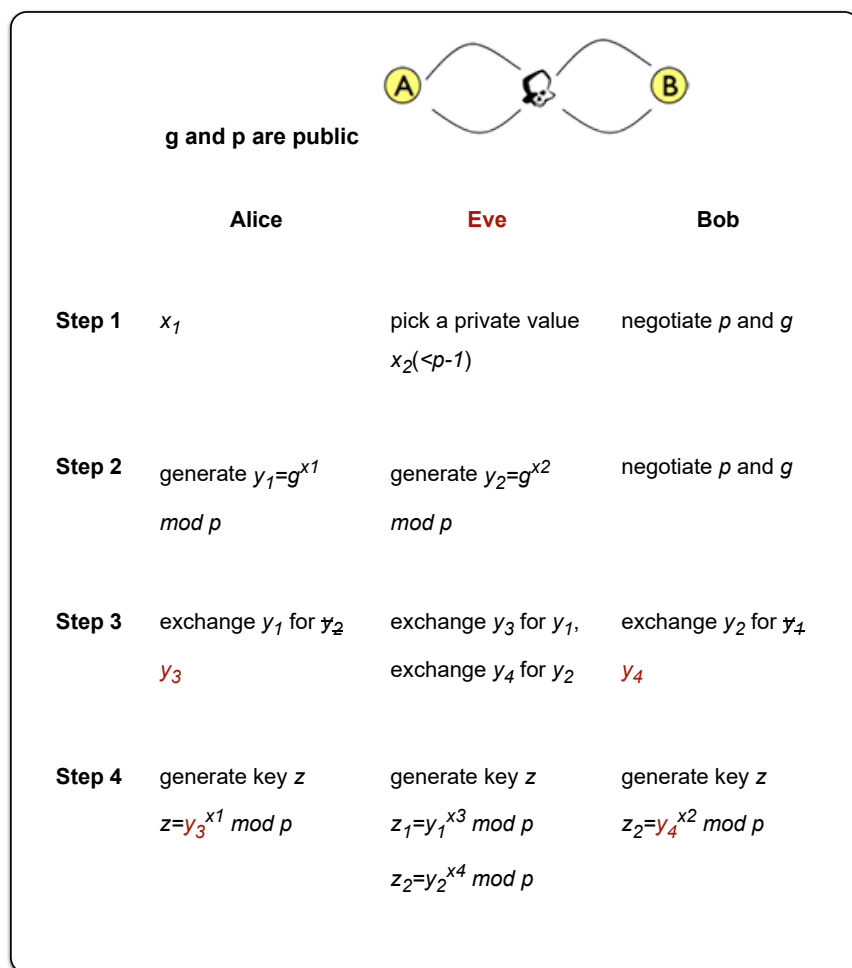
Step 3: Alice and Bob exchange y_1 and y_2

Step 4:

$$\text{Alice: } z = y_2^{z_1} \bmod p = 10^4 \bmod 23 = 18$$

$$\text{Bob: } z = y_1^{x_2} \bmod p = 4^3 \bmod 23 = 18$$

However, DHKE protocol suffers from the man in the middle (MITM) attack, since DHKE deals with key agreement, without authentication. For instance, Eve can jump in the middle between Alice and Bob, claiming to be Bob to Alice and claiming to be Alice to Bob. Therefore, both Alice and Bob think they are talking directly to each other, and neither of them really knows anything about who they have exchanged keys with. As illustrated in the figure below, initially, Alice and Bob still negotiate g and p and exchange them via the communication channel, so Eve also learns g and p . Then, Eve prepares x_3 and x_4 , and computes y_3 and y_4 when Alice computes y_1 and Bob computes y_2 . Next, when Alice sends y_1 to Bob and Bob sends y_2 to Alice, Eve intercepts y_1 and y_2 , but sends y_3 to Alice and y_4 to Bob. From Alice's point of view, she believes y_3 was from Bob, so she still computes z_1 based on it. With y_1 from Alice, Eve is able to compute the same z_1 as Alice. Importantly, Alice believes z_1 is the shared key negotiated with Bob via DHKE, so she will use it to encrypt messages to Bob. With z_1 at hand, Eve will be able to decrypt all the messages from Alice to Bob. Similarly, Eve also computes the same z_2 as Bob, and will be able to intercept and decrypt all the messages from Bob to Alice.



RSA (Rivest-Shamir-Adleman)

RSA is a public key cryptosystem proposed by Ron Rivest, Adi Shamir, and Leonard Adleman in 1977. The name RSA comes from the last names of the three authors. It is asymmetric, since encryption and decryption use different keys. The RSA algorithm is conceptually simple, but proving why it is secure involves deep mathematics (number theory). As we did for DHKE, we will just focus on how it can be used, rather than the underlying math.



Each party needs to generate a pair of a public key and a private key based on RSA, as illustrated in the example below. Note that for Step 4, two integers are relatively prime if the only positive integer that can be evenly divided by them is 1, i.e., their greatest common divisor (GCD) is 1. For example, 7 and 20 are relatively prime since they can only be evenly divided by 1, but 5 and 20 are not since both of them can be evenly divided by 5.

- | | | |
|---------------|--|---------------------------------------|
| Step 1 | Pick two large prime numbers p and q | $p=3, q=11$ |
| Step 2 | Calculate n and pq | $n=p*q=3*11= 33$ |
| Step 3 | Calculate $\phi(n) = (p-1)(q-1)$ | $\phi(n)=(3-1)(11-1)=20$ |
| Step 4 | Pick e such that it is relatively prime to $\phi(n)$ | $e=7$, so $\text{GCD}(e, \phi(n))=1$ |
| Step 5 | Pick d such that $d*e \bmod \phi(n) = 1$ | $d=3$, so $d*e \bmod 20=1$ |
| | Public key $k^+ = \{e, n\}$ | Public key $k^+ = \{7, 20\}$ |
| | Private key $k^- = \{d, n\}$ | Private key $k^- = \{3, 20\}$ |

Then, the encryption and decryption can be written as below:

$$E(k^+, P): \text{ciphertext} = \text{plaintext}^e \bmod n$$

$$D(k^-, C): \text{plaintext} = \text{ciphertext}^d \bmod n$$

Test Yourself

Encrypt the following data and then decrypt it based on RSA.

Public key: $(7, 33)$, Private key: $(3, 33)$

Data: 4 (encoding of the actual data)

$$E(k^+, P) = E(\{7, 33\}, 4) = 4^7 \bmod 33 = 16384 \bmod 33 = 16$$

$$D(k^-, P) = D(\{7, 33\}, 16) = 16^3 \bmod 33 = 4096 \bmod 33 = 4$$

Everyone using RSA needs to expose their public keys, but keep their private key confidential. Still consider the communication between Alice and Bob using RSA. Since everyone exposed their public key, Alice knows Bob's public key, K_B^+ , and Bob knows

Alice's public key, K_A^+ . Whenever Alice would like to send a message, P , to Bob, she needs to encrypt it using Bob's public key, i.e., $C = E(K_B^+, P)$, and send C to Bob. When receiving C , Bob uses his own private key, K_B^- , to decrypt it, i.e., $P = D(K_B^-, C)$, to obtain the plaintext message, P . Since only Bob knows his private key, no one else could decrypt C to get P . Then, Bob can use Alice's public key, K_A^+ , to encrypt the response message, and send it to Alice, who uses her private key, K_A^- , to decrypt the response.

Most of the time, one uses the recipient's public key to encrypt plaintext messages, which ensures only the recipient, who knows their own private key, can decrypt it. However, there exists a scenario where one uses their own private key to encrypt a message, which is called a digital signature. Think about signatures in the physical world, which assert a document's authenticity and non-repudiation. Similarly, we need digital signatures to assert the authenticity and non-repudiation of messages. The former means no one else can forge the message, and the latter means the sender can never disavow sending the message. For instance, to sign a document, d , digitally, Alice uses her private key, K_A^- , to encrypt it as the digital signature, i.e., $S(d) = E(K_A^-, d)$. For anyone else to validate the signature, they just use Alice's public key, K_A^+ , to decrypt the signature, i.e., $D(K_A^+, S(d))$. If the decryption gives d , Alice's signature is validated. The rationale is that since Alice is the only one who knows her private key, K_A^- , successful validation of the signature using Alice's public key indicates the signature must be obtained by encryption using Alice's private key.

■ Topic 2 Hash and Keyed Hash

Introduction

The hash function takes input of variable length and produces a fixed length value, called the hash or digest. We use $h()$ to indicate the hash function. A [hash table](#) is an example of using a hash function to map inputs into an array of slots.

Message Digest

Data communication is prone to errors during transmission. We all have experienced a mobile phone call that skips or distorts a small segment of conversation, or television/radio signal problems. Such transmission errors can also happen for computer networks, but the problem is how we can detect if the data is complete and intact after transmission. The message digest can help solve this problem to some extent. Consider a very simple digest, parity bit, computed by summing all bits of the messages using modulo 2. As illustrated below, with the parity bit as the message digest, Bob can compute the parity bit based on the data (1001) he receives, and compare it with the received parity bit (0) to detect any transmission error. For instance, if Bob receives 10000, then the parity bit he computes is $(1 + 0 + 0 + 0) \bmod 2 = 1$, which does not match the parity bit he receives from Alice. Hence, Bob knows the received data is corrupted.



Alice wants to send 1001
 Alice computes parity bit: $(1 + 0 + 0 + 1) \bmod 2 = 0$
 Alice appends parity bit and sends Bob: 10010



Bob receives 10010
 Bob computes parity bit: $(1 + 0 + 0 + 1) \bmod 2 = 0$
 Bob compares his computed parity with the received parity

Test Yourself

You may have noticed that the parity bit is quite limited in detecting transmission errors. What if Bob receives 10100? Can Bob detect whether data has been corrupted during transmission?

No.

Bob computes the parity bit by himself: $(1 + 0 + 1 + 0) \bmod 2 = 0$.

Bob compares it with the received parity bit: $0 = 0$. Bob does not know the data he received (1010) is not the same as the one Alice sent (1001).

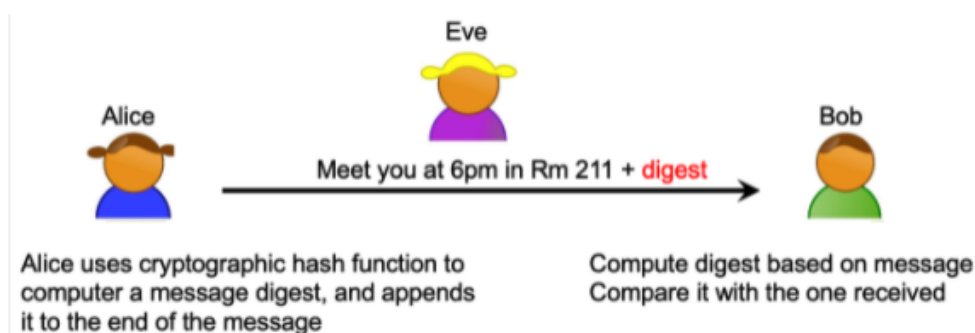
Simple hash functions such as the parity bit, modulo, and many others can only guarantee one thing: two identical outputs will be produced for two identical inputs, but there is no guarantee that two identical outputs imply the inputs are the same (as in the parity bit example above). Therefore, they are quite limited in detecting transmission errors when used to produce a message digest.

Cryptographic Hash Functions

Cryptographic hash functions have much stronger properties compared to simple hash functions. A good cryptographic hash function should demonstrate two fundamental properties: it is a one-way function and is collision resistant. The former means that it should be computationally hard to invert the hash function, i.e., computationally hard to compute d given y where $y = h(d)$. The latter means it should be computationally hard to find two different messages with the same hash, i.e., computationally hard to find x_1 and x_2 that $h(x_1) = h(x_2)$. There are a number of frequently used cryptographic hash functions, such as MD4, MD5, SHA1, SHA2, SHA3, etc., with MD short for message digest and SHA short for secure hash algorithm. Refer to our course textbook (Pfleeger et al., 2015) page 800 for a detailed description of them. Note that practical attacks against MD4/5 and SHA1 have been reported by researchers recently.

MAC and HMAC

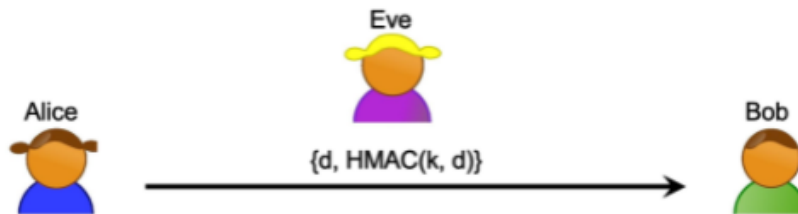
For security, we care about data authenticity, i.e., the ability to determine if anyone (e.g., the attacker Eve) forged or maliciously modified a message. In particular, Alice sends a message together with its digest to Bob, hoping Bob is able to determine if the message has been compromised by Eve or not.



However, even using a cryptographic hash function to produce a digest cannot ensure data authenticity. Suppose Alice uses a cryptographic hash function $h()$ to compute the digest over the data d , and sends $\{d, h(d)\}$ to Bob. Bob verifies its authenticity by computing $h(d)$ based on the received d by himself and then compares it with the received $h(d)$. The problem is any hash function is public. Eve can intercept $(d, h(d))$, revise d to become d' and compute $h(d')$ by herself. Then, Eve just sends $\{d', h(d')\}$ to Bob, who cannot detect the received data d' actually has been modified, since the computed hash matches the received hash.

To address the above problem, we need to consider keyed hash, e.g., the message authentication code (MAC). MAC takes a message, d , of variable length and a key, k , of fixed length as inputs and produces the MAC value, e.g., $MAC(k, d) = h(k \text{ XOR } d)$.

Without knowing the key k , Eve cannot forge a legitimate MAC for a message d . Hence, if Alice and Bob share a secret key k , Alice can send $\{d, \text{MAC}(k, d)\}$ to Bob. Eve can still modify d to become d' , but cannot compute $\text{MAC}(k, d')$ without knowing k . However, MAC is found to suffer from length extension attacks (for more details, refer to [Length Extension Attack](#)). Basically, Eve can compute $\text{MAC}(k, d+d')$ based on $\text{MAC}(k, d)$ even if she does not know the key k , where d' can be arbitrary data appended after the original data d . Therefore, by intercepting $\{d, \text{MAC}(k, d)\}$ from Alice, Eve can forge $\{d+d', \text{MAC}(k, d+d')\}$ and send it to Bob, who is not capable of detecting such a compromise. One solution is to use HMAC, i.e., hash based MAC, $\text{HMAC}(k, d) = h(k \text{ XOR } \text{MAC}(k, d))$, which is immune to length extension attacks. To ensure data authenticity, Alice can send $\{d, \text{HMAC}(k, d)\}$ to Bob, who can always verify if d has been corrupted or not.



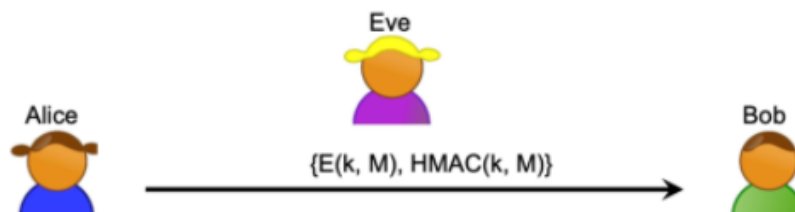
■ Topic 3 Data Authenticity and Confidentiality

Introduction

With $\{d, \text{HMAC}(k, d)\}$, we can ensure data authenticity, but not confidentiality, since the data, d , is sent in plaintext. To ensure the confidentiality of d , we need to encrypt d as well. Therefore, to ensure confidentiality and authenticity, the message sent from Alice to Bob should follow the format of $\{\text{encrypted plaintext}, \text{message digest}\}$.

Using Symmetric Key Encryption

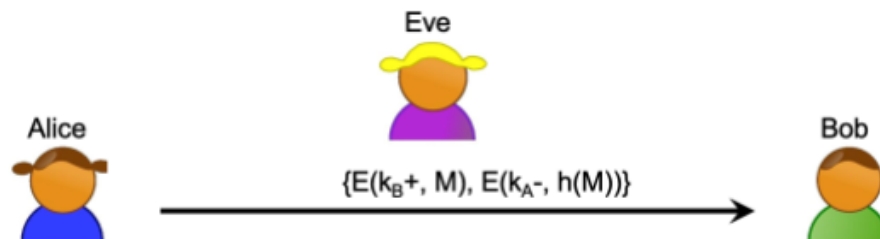
Alice and Bob already share a key, k , which might have been negotiated using DHKE. Alice wants to send a message, M , to Bob, and ensure both confidentiality and integrity (authenticity). To ensure data confidentiality, M cannot be sent in plaintext. Instead, Alice uses the key k shared with Bob to encrypt it, i.e., $E(k, M)$. To ensure data authenticity, Alice also computes $\text{HMAC}(k, M)$ and appends it after the ciphertext. Hence, what is sent by Alice to Bob is $\{E(k, M), \text{HMAC}(k, M)\}$. When Bob receives the message, he uses the shared key k to decrypt $E(k, M)$, thus obtaining M , based on which he computes $\text{HMAC}(k, M)$ by himself. Bob verifies the data authenticity by comparing the HMAC he computed and the HMAC received from Alice. Without the key k , Eve does not know how to decrypt $E(k, M)$ to get M even though she can observe $\{E(k, M), \text{HMAC}(k, M)\}$. Meanwhile, revising any part of $E(k, M)$ or $\text{HMAC}(k, M)$ results in failed verification on Bob's end.



Using Asymmetric Key Encryption

Alice and Bob know each other's public key. Alice wants to send a message, M , to Bob, and ensure both confidentiality and integrity (authenticity). To ensure data confidentiality, M cannot be sent in plaintext. Instead, Alice uses Bob's public key, k_B^+ , to encrypt it, i.e., $E(k_B^+, M)$. To ensure data authenticity, Alice can sign on M , i.e., $E(k_A^-, M)$, so Bob can validate her signature. However, there is a problem with sending the signature directly to Bob. Because everyone, including Eve knows Alice's public key, anyone can simply decrypt the signature $D(k_A^+, E(k_A^-, M))$ to obtain the plaintext M , thus compromising data confidentiality. To address this problem, Alice's signature can be improved a bit to become $E(k_A^-, h(M))$, i.e., **Alice signs on the hash of M** . Even if such a signature could be decrypted by Eve, she would only learn the hash of M . Due to the one-way property of the hash function, Eve cannot learn the plaintext M . Note that the hash of M , i.e., $h(M)$, is enough for Bob to verify data authenticity.

Hence, what is sent by Alice to Bob is $\{E(k_B^+, M), E(k_A^-, h(M))\}$. When Bob receives the message, he first uses his private key, k_B^- , to decrypt $E(k_B^+, M)$, thus obtaining the plaintext M , based on which he computes the hash of M by himself. Then, Bob uses Alice's public key, k_A^+ , to validate the signature by decrypting $E(k_A^-, h(M))$, thus getting $h(M)$. Bob verifies data authenticity by comparing the $h(M)$ he computed and the $h(M)$ decrypted from Alice's signature. Without knowing Bob's private key, k_B^- , Eve does not know how to decrypt $E(k_B^+, M)$ to get plaintext M even though she can observe $\{E(k_B^+, M), E(k_A^-, h(M))\}$. Meanwhile, decrypting $E(k_A^-, h(M))$ using Alice's public key only reveals the hash of M , rather than M itself. Also, revising any part of $E(k_B^+, M)$ or $E(k_A^-, h(M))$ results in failed verification on Bob's end.



Using Session Key

In reality, there might be a large number of messages transmitted between Alice and Bob. (Consider the communication between a web browser and a web server, for example.) Using the same key to encrypt many messages significantly increases the possibility of attackers guessing the key. At the same time, switching to a new key frequently is always costly for both symmetric key encryption (using DHKE) and asymmetric key encryption (using RSA). A practical, widely used solution is the session key, which is used in one session only and expires after the session.

To begin with, Alice and Bob know each other's public key. They first negotiate a session key, k_x . For example, Alice picks up k_x and sends it to Bob. Of course, it cannot be sent in plaintext. Alice would like to ensure both the confidentiality and authenticity of the key, so she uses asymmetric key encryption (e.g., public key encryption) to transmit the key to Bob, i.e., $\{E(k_B^+, k_x), E(k_A^-, h(k_x))\}$. Bob can use his private key to decrypt and learn k_x , and validate it using Alice's public key to decrypt the signature (just as in the above asymmetric key encryption by replacing M with k_x). Now both Alice and Bob know the session key, k_x , so they begin to use symmetric key encryption to transmit messages in the current session. When the current session ends, k_x expires and is dropped. If a new session begins, Alice and Bob need to negotiate a new session key and use it to encrypt/decrypt messages in the new session.

Test Yourself

What should be sent from Alice to Bob for data d using the session key k_x to ensure both confidentiality and authenticity?

$\{E(k_x, d), \text{HMAC}(k_x, d)\}$

References

- Pfleeger, C. P., Pfleeger, S. L., & Marguiles, J. (2015). *Security in computing* (5th ed.). Prentice Hall.
- Stallings, W. (2011). *Cryptography and network security: Principles and practice* (5th ed.). Pearson.
- Van Houtven, L. (2017). [Crypto 101](#).
- Hoffstein, J., Pipher, J., & Silverman, J. H. (2014). *An introduction to mathematical cryptography* (2nd ed.). Springer.

Boston University Metropolitan College